LAPORAN REFLEKSI
PEMROGRAMAN BERORIENTASI OBJEK
IMPLEMENTASI SISTEM RESERVASI HOTEL

Dosen Pengampu: Wayan Oger Vihikan, S.T.I, M.I.T.

Disusun oleh:

Ngurah Made Arya Mahardi Nata (2505551160)

Ni Putu Niniek Anjaswari (2505551170)

Pemrograman Berorientasi Objek (B)

PROGRAM STUDI SARJANA TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS UDAYANA

2026

1. AI tool apa yang digunakan dan mengapa memilih tool tersebut?

Kelompok kami menggunakan AI *tool* berupa *ChatGPT* untuk membantu proses pengembangan kode pada fase 2. Kami memilih *ChatGPT* karena mampu memberikan respon yang cepat, penjelasan yang mudah dipahami, serta dapat memberikan contoh kode yang relevan dengan permasalahan yang dihadapi.

Selain itu, *ChatGPT* juga membantu dalam melakukan *review* terhadap kode fase 1, sehingga kami dapat mengetahui bagian mana yang masih kurang sesuai dengan konsep *Object-Oriented Programming* (OOP).

2. Apa saja saran/perubahan yang diberikan oleh AI terhadap kode Fase 1?

Berdasarkan hasil diskusi dengan AI, terdapat beberapa saran perbaikan yang diberikan untuk meningkatkan kualitas kode fase 1 agar lebih sesuai dengan konsep Object-Oriented Programming (OOP). Perbaikan tersebut tidak hanya berfokus pada struktur kode, tetapi juga pada cara pengelolaan data dan interaksi antar class.

2.1 Encapsulation (private + getter)

AI menyarankan agar atribut pada *class* tidak diakses secara langsung, melainkan menggunakan prinsip *encapsulation* dengan mengubah atribut menjadi `private` dan menyediakan *method getter*.

Sebelum:

```
int harga;
```

Kode Program 2.1 Sebelum penggunaan *encapsulation*

Sesudah

```
private int harga;

public int getHarga() {
    return harga;
}
```

Kode Program 2.2 Setelah penggunaan *encapsulation*

2.2 Menghindari akses langsung atribut

AI juga menyarankan untuk menghindari akses langsung terhadap atribut dari luar *class* karena melanggar prinsip *encapsulation*.

Sebelum:

```
daftarKamar[i].harga
```

Kode Program 2.3 Sebelum menghindari akses langsung

Sesudah:

```
daftarKamar[i].getHarga()
```

Kode Program 2.4 Sesudah menghindari akses langsung

2.3 Perbaikan class Reservasi

AI menyarankan agar *class* Reservasi tidak hanya menerima nilai harga, tetapi menggunakan *object* agar mencerminkan hubungan antar entitas dalam program.

Sebelum:

```
class Reservasi {
    int total;

    Reservasi(int harga, int malam) {
        total = harga * malam;
    }
}
```

Kode Program 2.5 Sebelum perbaikan *class*

Sesudah:

```
class Reservasi {
    private Pelanggan pelanggan;
    private Kamar kamar;
    private int malam;

    public Reservasi(Pelanggan pelanggan, Kamar kamar, int
malam) {
        this.pelanggan = pelanggan;
        this.kamar = kamar;
        this.malam = malam;
    }

    public int hitungTotal() {
        return kamar.getHarga() * malam;
    }
}
```

Kode Program 2.6 Sesudah perbaikan *class*

2.4 Menu interaktif

AI menyarankan untuk menambahkan menu interaktif agar program tidak hanya berjalan sekali, tetapi dapat digunakan berulang kali sesuai pilihan pengguna.

```
int pilihan;

do {
    System.out.println("1. Lihat Kamar");
    System.out.println("2. Reservasi");
    System.out.println("0. Keluar");

    pilihan = input.nextInt();

    switch (pilihan) {
        case 1:
            // tampil kamar
            break;
        case 2:
            // reservasi
            break;
    }

} while (pilihan != 0);
```

Kode Program 2.7 Struktur kontrol *do-while* dengan percabangan *switch case* dalam menu

2.5 Validasi input

AI menyarankan untuk menambahkan validasi *input* agar program tidak mengalami *error* ketika pengguna memasukkan data yang tidak sesuai.

```
while (!input.hasNextInt()) {
    System.out.println("Harus angka!");
    input.next();
}
int pilih = input.nextInt();
```

Kode Program 2.8 Contoh penambahan validasi input pada program

2.6 Perbaikan output (printf)

AI juga memberikan saran untuk merapikan tampilan *output* menggunakan `printf` agar hasil lebih terstruktur dan mudah dibaca.

```
System.out.printf("%-5d %-10d %-10s Rp%-10d%n",
    (i + 1),
    kamar.getNomor(),
```

```
    kamar.getTipe(),
    kamar.getHarga()
);
```

Kode Program 2.9 Perbaikan output pada *class* reservasi

2.7 Fitur tambahan (check-in & check-out)

Selain itu, AI menyarankan penambahan fitur seperti *check-in* dan *check-out* untuk membuat program lebih realistis dan tidak hanya berfungsi sebagai perhitungan total biaya.

```
boolean sudahCheckIn = false;

void checkIn() {
    sudahCheckIn = true;
    System.out.println("Berhasil check-in");
}

void checkOut() {
    if (sudahCheckIn) {
        System.out.println("Check-out berhasil");
    } else {
        System.out.println("Belum check-in");
    }
}
```

Kode Program 2.10 Penambahan fitur *check in* dan *check out*

3. Saran mana yang kalian terima dan mengapa?

Kami menerima sebagian besar saran yang diberikan oleh AI karena saran tersebut sesuai dengan konsep dasar *Object-Oriented Programming* (OOP) dan materi yang telah dipelajari di perkuliahan.

Salah satu saran utama yang kami terima adalah penerapan *encapsulation*, yaitu dengan mengubah atribut menjadi `private` dan menggunakan method getter. Hal ini membuat data dalam *class* menjadi lebih aman dan tidak dapat diakses secara langsung dari luar.

Selain itu, kami juga menerima saran terkait perbaikan desain *class* Reservasi dengan menggunakan *object* `Pelanggan` dan `Kamar`. Perubahan ini membuat hubungan antar *class* menjadi lebih jelas dan mencerminkan kondisi nyata dalam sistem reservasi hotel.

Kami juga menerapkan saran untuk menambahkan menu interaktif dan validasi *input*. Menu interaktif membuat program dapat digunakan berulang kali, sedangkan validasi *input* mencegah terjadinya *error* saat program dijalankan. Perbaikan output menggunakan `printf` juga kami terima karena membuat tampilan program lebih rapi dan mudah dibaca.

4. Saran mana yang kalian tolak dan mengapa?

Tidak semua saran yang diberikan oleh AI kami terapkan secara langsung. Beberapa saran kami pertimbangkan kembali agar tetap sesuai dengan ruang lingkup tugas dan materi yang telah dipelajari.

Sebagai contoh, terdapat saran yang mengarah pada pengembangan program yang lebih kompleks, seperti penggunaan konsep *Object-Oriented Programming* lanjutan. Namun, kami memilih untuk tidak menerapkan konsep tersebut karena belum dibahas secara mendalam dalam perkuliahan.

Kami juga mempertimbangkan agar program tetap sederhana dan mudah dipahami, sehingga tidak menambahkan fitur yang terlalu kompleks di luar kebutuhan tugas. Dengan demikian, kami hanya memilih saran yang relevan dan sesuai dengan tujuan pembelajaran.

5. Apakah ada saran dari AI yang salah atau kurang tepat? Jika ya, jelaskan.

Secara umum, saran yang diberikan oleh AI sudah tepat dan membantu dalam memperbaiki kode program. Namun, terdapat beberapa saran yang perlu disesuaikan dengan konteks tugas dan materi pembelajaran.

Beberapa saran yang diberikan cenderung mengarah pada pengembangan yang lebih kompleks dari yang dibutuhkan. Oleh karena itu, kami melakukan penyederhanaan agar tetap sesuai dengan tingkat pemahaman dan materi yang telah dipelajari di kelas. Dengan kata lain, saran dari AI tidak sepenuhnya salah, tetapi perlu disesuaikan agar tetap relevan dengan kebutuhan tugas.

6. Konsep OOP apa yang kalian pahami lebih baik setelah proses review dengan AI?

Setelah melakukan proses review dengan AI, kami menjadi lebih memahami beberapa konsep dasar *Object-Oriented Programming* (OOP). Salah satu konsep yang lebih kami pahami adalah *encapsulation*, yaitu pentingnya menyembunyikan data dalam *class* menggunakan *private* dan mengaksesnya melalui *method getter*. Kami juga lebih memahami bagaimana menghindari akses langsung terhadap atribut.

Selain itu, kami memahami konsep relasi antar *object*, yaitu bagaimana *class* seperti Pelanggan, Kamar, dan Reservasi saling berhubungan dalam sebuah sistem. Hal ini membuat desain program menjadi lebih terstruktur dan mendekati kondisi nyata. Kami juga memahami bahwa *class* tidak hanya berfungsi untuk menyimpan data, tetapi juga memiliki method sebagai perilaku (*behavior*). Dengan demikian, program menjadi lebih terorganisir dan sesuai dengan prinsip OOP.

7. Pembagian tugas antar anggota kelompok

Pembagian tugas dalam kelompok ini dilakukan secara kolaboratif dengan mempertimbangkan kemampuan dan peran masing-masing anggota. Setiap anggota memiliki tanggung jawab pada setiap tahap pengerjaan, mulai dari pengembangan program pada fase 1 dan fase 2, penyusunan laporan refleksi, hingga dokumentasi log AI. Dengan adanya pembagian tugas ini, diharapkan proses pengerjaan dapat berjalan secara terstruktur, efektif, dan menghasilkan output yang optimal.

Tabel 7.1 Pembagian Tugas Kelompok

Nama	**NIM**	**Tugas**
Ni Putu Niniek Anjaswari	2505551170	Bertugas menyusun struktur class Pelanggan, Kamar, dan Reservasi serta membantu dalam pengembangan program
Ni Putu Niniek Anjaswari	2505551170	Pengembangan fitur OOP (*encapsulation*, relasi *object*)

Ni Putu Niniek Anjaswari	2505551170	Bertugas menyusun laporan refleksi bagian 1 yang mencakup penggunaan AI, saran perbaikan, dan penerapan pada kode
Ngurah Made Arya Mahardi Nata	2505551160	Bertugas mengembangkan program utama pada class Hotel, mengintegrasikan seluruh *class*, serta melakukan pengujian program
Ngurah Made Arya Mahardi Nata	2505551160	Bertugas menambahkan menu interaktif, validasi *input*, serta melakukan perbaikan dan pengujian program
Ngurah Made Arya Mahardi Nata	2505551160	Bertugas menyusun laporan refleksi bagian 2 yang mencakup evaluasi saran, pemahaman konsep OOP, dan pembagian tugas
Ni Putu Niniek Anjaswari	2505551170	Bertugas melakukan *screenshot* diskusi AI
Ngurah Made Arya Mahardi Nata	2505551160	Bertugas mengumpulkan dan menyusun hasil *screenshot* diskusi AI